ENGR 102 Summer Bridge
Day 10 Instructor Key

Lectures 1-5 Recap, Direct Input, and Repetition Launch

Friday, July 17, 2026 • 50 points

Name		UIN		Section	

Boolean-operator convention. Python operators appear as `and`, `or`, and `not`. Their lowercase spelling is required in code.

Daily target — increasing independence

Connect input, state, Boolean decisions, and testing to a first complete while-loop program.

Allowed tools	input, int, float, arithmetic, comparisons, Boolean operators `and`, `or`, and `not`, if/elif/else, assignment, print, and a first while loop after the quiz
Support supplied	Variable names, a floor-validity rule, the movement goal, and named boundary tests are supplied.
You must decide	The conversion, validation branch order, continuation condition, direction decision, and one-floor update.
Scaffold level	Planning frame supplied; the complete elevator loop is student-authored.

Scoring and completion standard — 50 points

Evidence	Points	Secure work shows
Integrated trace	14	intermediate state, condition results, and exact output
Diagnosis and repair	12	actual, intended, cause, repair, and a boundary test when requested
Full program and tests	24	coherent executable logic, required output, and defended tests

Independence standard. You may use the supplied requirements and examples, but you must produce one connected solution. A collection of disconnected correct lines is not yet a complete program.

Begin with the trace, then use its evidence habits when you construct.

Task 1. Integrated trace — 14 points

Trace types, Boolean subexpressions, and branch order. Do not jump directly to the final line.

```
sample = "14"
offset = 5
value = int(sample) * 2 + offset
stable = 20 <= value <= 35 and value % 3 == 0

if not stable:
    status = "review"
elif value >= 30:
    status = "high"
else:
    status = "normal"

print(value, stable, status)
```

Your task

1. Compute value and identify its type. Then evaluate each side of the **and** expression used to compute stable.
2. State which branch assigns status and explain why the earlier branch does not execute.
3. Write the exact printed output, including the Boolean capitalization.

Answer and explanation

1. int(sample) is 14, so value is 14 * 2 + 5 = 33, an integer. The range check 20 <= 33 <= 35 is True, and 33 % 3 == 0 is True; therefore stable is True.
2. **not** stable is False, so the first branch is skipped. value >= 30 is True, so the elif branch assigns status = high. The else branch is not considered after that match.
3. Exact output: 33 True high

A final answer without the requested state evidence cannot earn full trace credit.

Task 2. Diagnose and repair — 12 points

The requirement is: approval requires training, and it also requires either a temperature from 15 through 30 inclusive or a supervisor override.

```
trained = False
temperature = 20
supervisor = False

approved = trained and temperature >= 15 or temperature <= 30 or supervisor
print(approved)
```

Your task

1. Show the actual Boolean result for the supplied values and explain why it violates the requirement.
2. Write one corrected Boolean assignment and defend its grouping with a test in which trained is False.

Answer and explanation

1. Python evaluates **and** before **or**. trained **and** temperature >= 15 is False, but temperature <= 30 is True, so the entire expression becomes True. That incorrectly approves an untrained person.
2. Correct assignment: approved = trained **and** (15 <= temperature <= 30 **or** supervisor). With trained = False, the left side of the outer **and** is False, so approval is False regardless of temperature or supervisor. This matches the stated requirement that training is always required.

Debugging evidence is complete only when the repair is tied to the stated defect and an expected result.

Task 3. Construct a complete program — 24 points across Pages 4-5

Program specification

- Read current_floor and desired_floor as integers.
- Valid floors are 1 through 10 inclusive. Print invalid if either input is outside that interval.
- If the valid floors are equal, print already there.
- Otherwise, use a while loop that changes current_floor by exactly one floor per pass until it equals desired_floor.
- Print each newly reached floor inside the loop. Do not jump directly to the destination.

Required planning evidence

1. Write the invalid-input Boolean expression and the branch order that protects the loop.
2. Write the continuation condition and both possible one-floor updates before coding.

Answer and explanation

```
current_floor = int(input())
desired_floor = int(input())

invalid = (current_floor < 1 or current_floor > 10 or
           desired_floor < 1 or desired_floor > 10)

if invalid:
    print("invalid")
elif current_floor == desired_floor:
    print("already there")
else:
    while current_floor != desired_floor:
        if current_floor < desired_floor:
            current_floor = current_floor + 1
        else:
            current_floor = current_floor - 1
        print(current_floor)
```

Validation and the equal-floor case occur before the loop, so every loop pass begins with two valid unequal floors.

The continuation condition describes unfinished movement. Exactly one branch changes current_floor by one, which guarantees progress toward desired_floor from either direction.

Task 3 continued. Test and defend the program **included in 24 points**

Expected tests and results

1. Inputs 2 and 5 print 3, 4, and 5 on separate lines.
2. Inputs 5 and 2 print 4, 3, and 2 on separate lines. This protects the downward update.
3. Inputs 4 and 4 print already there. Inputs 0 and 4 print invalid.

Scoring guidance

4 points planning; 4 input/validation; 4 equal-floor branch; 4 while condition; 4 directional one-floor update; 2 output placement; 2 tests. Equivalent coherent code earns full credit.

Accept alternative correct code when it obeys the stated tool boundary, handles the required cases, and produces the required output. All blue text is answer or scoring guidance.

VIP Lab Alignment

Bridge Day 10 • Contract 2d4127aac856

This page adds a current lab handoff while preserving the workbook's independence-ramp tasks.

Why this page is here

VIP Lab Day(s): 5 **Activities:** 18.19, 18.21, 18.22

Begin with the notebook's required read-convert-compute-format-print reconnect, then transfer the elevator loop's continuation, update, and termination evidence into three while-loop activities.

Open these current notebook cells

Activity / stable cells	Notebook work	Evidence to carry forward
18.19 18-19-work / 18-19-transfer / 18-19-cold	**Countdown To Multiple**	Write a while condition from a divisibility goal.
18.21 18-21-work / 18-21-transfer / 18-21-cold	**Cooldown Sequence**	Track a current value and a stored sequence.
18.22 18-22-work / 18-22-transfer / 18-22-cold	**Rounds To Clear Queue**	Model repeated queue reduction.

Readiness check before you code

1. Write the five-part program shell, then for each mapped activity identify the input conversion, changing state, and continuation condition.

   Answer and explanation: The shell is input, numeric conversion, current-day computation, f-string formatting, and print. Numeric text becomes int or float before arithmetic. Activity 18.19 changes the current number until divisibility is reached; 18.21 changes the cooldown value while storing each state; 18.22 reduces backlog while counting rounds.

2. Explain in one sentence why the update moves the state toward termination.

   Answer and explanation: A complete explanation connects the direction and size of the update to the condition becoming false; merely saying that the loop eventually stops is insufficient.

Notebook and submission procedure

1. Use the sample and transfer cells for formative practice; attempt before opening a reveal.
2. Complete the end-of-notebook Independent Program Studio without a reveal or copied solution. Only the Studio cells listed above are scored for independent mastery in the matching Lab Day notebook.
3. Save or download the completed notebook as one `.ipynb` file.
4. Upload that one notebook to the matching Gradescope assignment. Do not export a `.py` file or create a ZIP.